

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph dan Tree

Oleh:

Putri Maiza Azzura NIM. 2510817220012

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph dan Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Putri Maiza Azzura
NIM : 2510817220012

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhammad Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	ii
DAFTAR ISI	iii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL	v
Soal 1: Konversi Adjacency List ke Adjacency Matrix	6
A. Source Code.....	8
B. Output Program	9
C. Pembahasan	9
Soal 2: Konversi Adjacency Matrix ke Adjacency List	12
A. Source Code.....	13
B. Output Program	15
C. Pembahasan	15
Soal 3: BFS: Jarak Terpendek Keluar Labirin.....	18
A. Source Code.....	20
B. Output Program	21
C. Pembahasan	21
Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin.....	24
A. Source Code.....	26
B. Output Program	27
C. Pembahasan	27
Soal 5: Tree Traversal.....	30
A. Source Code.....	31
B. Output Program	40
C. Pembahasan	40
TAUTAN GIT	43

DAFTAR GAMBAR

Gambar 1 Screenshot Hasil Output Soal 1	9
Gambar 2 Screenshot Hasil Output Soal 2	15
Gambar 3 Screenshot Hasil Output Soal 3	21
Gambar 4 Screenshot Hasil Output Soal 4	27
Gambar 2 Screenshot Hasil Output Soal 5	40

DAFTAR TABEL

Table 1 Source Code soal1.cpp	8
Table 2 Source Code soal2.cpp	13
Table 3 Source Code soal3.cpp	20
Table 4 Source Code soal4.cpp	26
Table 5 Source Code soal5.cpp	31
Table 6 Source Code RedBlackTree.h	33
Table 7 Source Code RedBlackTree.cpp.....	35

Soal 1: Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

M

$U_1 V_1 W_1$

$U_2 V_2 W_2$

...

$U_M V_M W_M$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .

- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

V1 V2 ... VN

V1 a11 a12 ... a1N

V2 a21 a22 ... a2N

...

VN aN1 aN2 ... aNN

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai

a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki self-loop.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6

C B 1	D 0 0 3 0 0
C E 6	E 0 0 0 0 0
D C 3	

A. Source Code

Table 1 Source Code soal1.cpp

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include <map>
5  using namespace std;
6
7  int main() {
8      int N;
9      cin >> N;
10
11     vector<string> labels(N);
12     map<string, int> idx;
13
14     for (int i = 0; i < N; i++) {
15         cin >> labels[i];
16         idx[labels[i]] = i;
17     }
18
19     vector<vector<int>> matrix(N, vector<int>(N, 0));
20
21     int M;
22     cin >> M;
23
24     for (int i = 0; i < M; i++) {
25         string u, v;
26         int w;
27         cin >> u >> v >> w;
28         matrix[idx[u]][idx[v]] = w;
29     }
30
31     cout << "Adjacency Matrix:" << endl;
32     cout << "  ";
33     for (int i = 0; i < N; i++) {
34         cout << " " << labels[i];
35     }
36     cout << endl;
37
38     for (int i = 0; i < N; i++) {
39         cout << labels[i];
40         for (int j = 0; j < N; j++) {

```

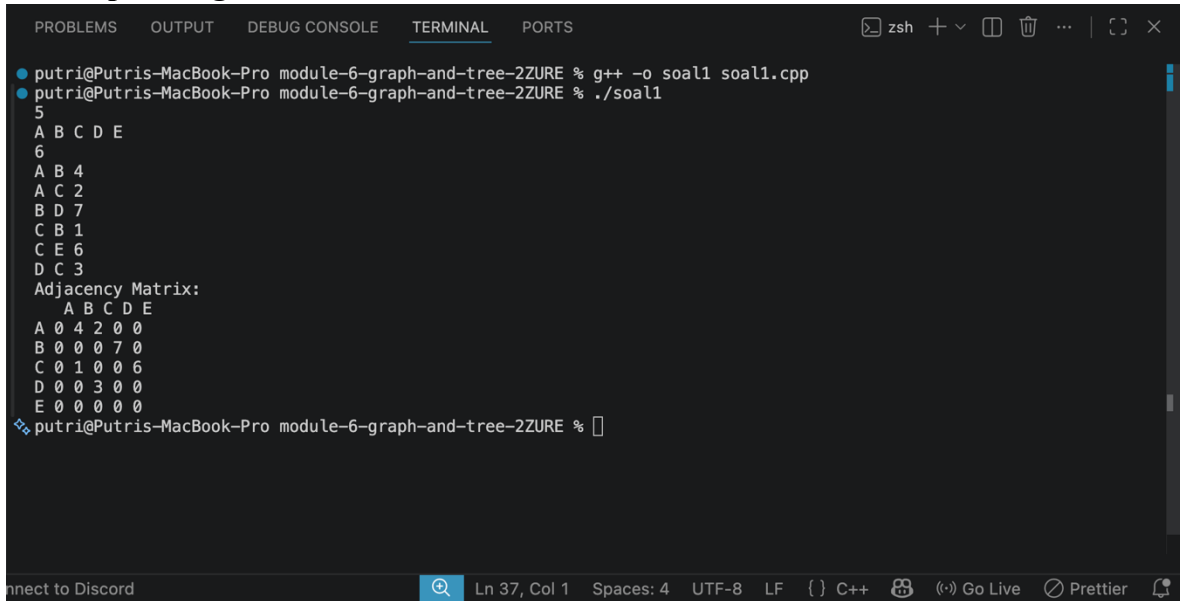


```

41         cout << " " << matrix[i][j];
42     }
43     cout << endl;
44 }
45
46 return 0;
47 }

```

B. Output Program



```

putri@Putris-MacBook-Pro module-6-graph-and-tree-2ZURE % g++ -o soal1 soal1.cpp
putri@Putris-MacBook-Pro module-6-graph-and-tree-2ZURE % ./soal1
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
  A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
putri@Putris-MacBook-Pro module-6-graph-and-tree-2ZURE %

```

Gambar 1 Screenshot Hasil Output Soal 1

C. Pembahasan

Algoritma yang digunakan pada program ini bertujuan untuk mengubah representasi graph berarah berbobot (directed weighted graph) yang diberikan dalam bentuk daftar edge menjadi bentuk adjacency matrix. Adjacency matrix merupakan representasi graph menggunakan matriks berukuran $N \times N$, di mana setiap baris dan kolom mewakili sebuah vertex. Jika terdapat edge dari suatu vertex menuju vertex lain, maka nilai pada sel matriks yang bersesuaian akan berisi bobot edge tersebut. Sebaliknya, jika tidak terdapat edge, maka nilainya adalah 0. Karena graph bersifat berarah, nilai pada posisi (i, j) tidak harus sama dengan posisi (j, i) .

Teknik yang digunakan dalam program ini adalah pemetaan label vertex ke indeks matriks menggunakan struktur data map. Setiap label vertex yang dibaca dari input disimpan ke dalam sebuah vector untuk mempertahankan urutan vertex, kemudian dimasukkan ke dalam

map agar setiap label memiliki indeks numerik yang sesuai. Setelah itu, program membuat adjacency matrix berukuran $N \times N$ dan menginisialisasi seluruh elemennya dengan nilai 0. Langkah ini dilakukan karena sesuai spesifikasi soal, nilai 0 menunjukkan bahwa tidak terdapat edge antara dua vertex.

Cara kerja algoritma dimulai dengan membaca jumlah vertex dan label setiap vertex. Selanjutnya program membaca jumlah edge yang ada pada graph. Untuk setiap edge yang dibaca dalam format vertex asal, vertex tujuan, dan bobot, program mencari indeks kedua vertex tersebut melalui map. Setelah indeks ditemukan, bobot edge langsung disimpan ke posisi matriks yang sesuai, yaitu pada baris vertex asal dan kolom vertex tujuan. Dengan cara ini, setiap edge yang terdapat pada input akan direpresentasikan secara langsung ke dalam adjacency matrix. Setelah seluruh edge diproses, program mencetak matriks beserta label vertex sesuai format keluaran yang diminta.

Program dibuat berdasarkan beberapa asumsi yang telah dijamin oleh soal. Semua label vertex bersifat unik sehingga setiap vertex dapat dipetakan ke indeks yang berbeda. Selain itu graph tidak memiliki self-loop maupun edge duplikat dengan arah yang sama. Bobot edge selalu bernilai positif, sedangkan nilai 0 digunakan untuk menunjukkan tidak adanya hubungan antarvertex. Karena seluruh kondisi tersebut telah dijamin pada input, program tidak perlu melakukan validasi tambahan terhadap data yang diterima.

Dari sisi kompleksitas, proses pembacaan vertex membutuhkan waktu sebesar $O(N)$ karena dilakukan satu kali untuk setiap vertex. Pembacaan seluruh edge membutuhkan kompleksitas $O(M \log N)$ karena setiap pencarian indeks vertex pada struktur map memerlukan waktu $O(\log N)$ dan dilakukan sebanyak M kali. Selanjutnya, proses pencetakan adjacency matrix membutuhkan waktu $O(N^2)$ karena seluruh elemen matriks harus ditampilkan. Dengan demikian, kompleksitas waktu total algoritma adalah $O(N^2 + M \log N)$. Untuk kompleksitas ruang, program menyimpan label vertex, struktur map, dan adjacency matrix. Karena adjacency matrix berukuran $N \times N$, kebutuhan ruang didominasi oleh matriks tersebut sehingga kompleksitas ruang keseluruhan adalah $O(N^2)$.

Berdasarkan permasalahan yang diberikan, algoritma ini dapat dibuktikan benar karena setiap edge yang dibaca dari input akan ditempatkan tepat pada posisi matriks yang sesuai dengan vertex asal dan vertex tujuan. Seluruh elemen matriks diinisialisasi dengan nilai 0 sehingga pasangan vertex yang tidak memiliki edge akan tetap bernilai 0. Sementara itu, setiap edge yang ada akan mengubah nilai sel matriks menjadi bobot yang diberikan. Karena arah edge disimpan pada posisi baris asal dan kolom tujuan, sifat graph berarah tetap terjaga. Oleh sebab itu, adjacency matrix yang dihasilkan selalu merepresentasikan graph yang sama dengan daftar edge pada input sehingga solusi ini memenuhi seluruh ketentuan yang diminta oleh soal.

Soal 2: Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

$A_{11} A_{12} \dots A_{1N}$

$A_{21} A_{22} \dots A_{2N}$

...

$A_{N1} A_{N2} \dots A_{NN}$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

V1 -> (X1,w1), (X2,w2), ...

V2 -> (X1,w1), (X2,w2), ...

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Batasan

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4), (C,2) B -> (D,7) C -> (B,1), (E,6) D -> (C,3) E -> -

A. Source Code

Table 2 Source Code soal2.cpp

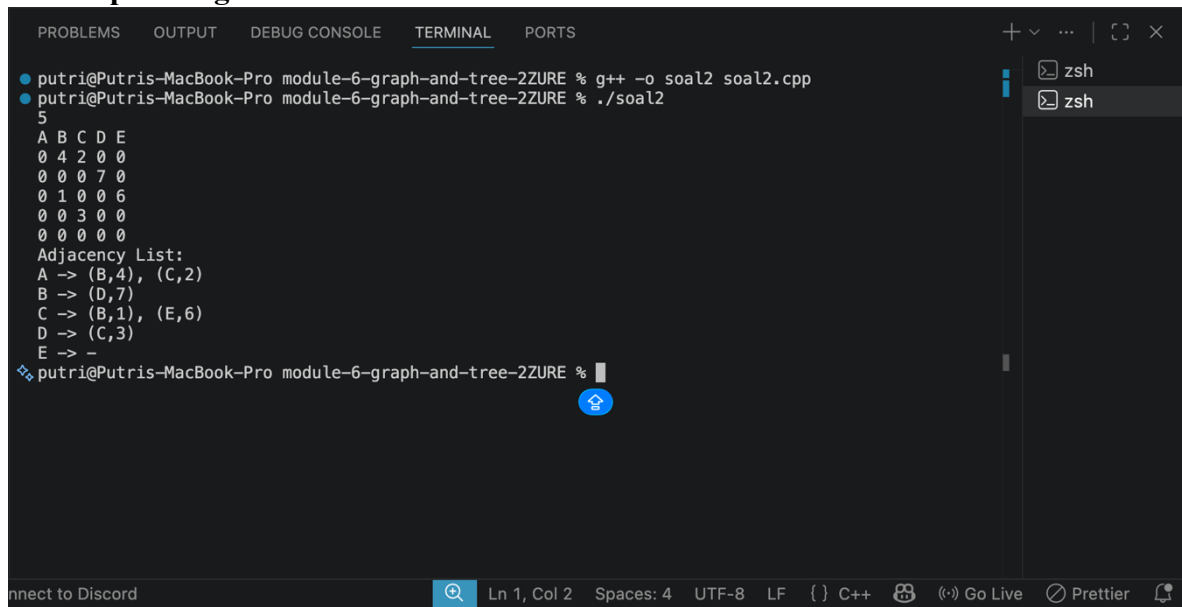
1	#include <iostream>
2	#include <string>

```

3  #include <vector>
4  using namespace std;
5
6  int main() {
7      int N;
8      cin >> N;
9
10     vector<string> labels(N);
11     for (int i = 0; i < N; i++) {
12         cin >> labels[i];
13     }
14
15     vector<vector<int>> matrix(N, vector<int>(N));
16     for (int i = 0; i < N; i++) {
17         for (int j = 0; j < N; j++) {
18             cin >> matrix[i][j];
19         }
20     }
21
22     cout << "Adjacency List:" << endl;
23     for (int i = 0; i < N; i++) {
24         cout << labels[i] << " -> ";
25
26         bool hasEdge = false;
27         bool first = true;
28         for (int j = 0; j < N; j++) {
29             if (matrix[i][j] > 0) {
30                 if (!first) cout << ", ";
31                 cout << "(" << labels[j] << ", " <<
32 matrix[i][j] << ")";
33                 first = false;
34                 hasEdge = true;
35             }
36         }
37
38         if (!hasEdge) {
39             cout << "-";
40         }
41         cout << endl;
42     }
43
44     return 0;
45 }

```

B. Output Program



```
putri@Putris-MacBook-Pro module-6-graph-and-tree-2ZURE % g++ -o soal2 soal2.cpp
putri@Putris-MacBook-Pro module-6-graph-and-tree-2ZURE % ./soal2
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
```

Gambar 2 Screenshot Hasil Output Soal 2

C. Pembahasan

Algoritma yang digunakan pada program ini bertujuan untuk mengubah representasi graph berarah berbobot dari bentuk adjacency matrix menjadi adjacency list. Adjacency list merupakan representasi graph yang menyimpan daftar vertex tujuan yang terhubung langsung dari setiap vertex beserta bobot edge-nya. Pada soal, nilai matriks yang lebih besar dari 0 menunjukkan adanya edge dari vertex asal menuju vertex tujuan dengan bobot tertentu, sedangkan nilai 0 menunjukkan tidak adanya edge. Oleh karena itu, program harus memeriksa setiap elemen matriks dan menampilkan hubungan yang valid dalam bentuk daftar ketetanggaan.

Teknik yang digunakan adalah traversal matriks dua dimensi (2D array traversal). Program menyimpan seluruh label vertex ke dalam sebuah vector agar urutan vertex tetap sesuai dengan input. Selanjutnya adjacency matrix disimpan dalam `vector<vector<int>>`. Setelah seluruh data berhasil dibaca, program melakukan penelusuran setiap baris matriks. Setiap baris merepresentasikan sebuah vertex asal, sedangkan setiap kolom merepresentasikan vertex tujuan. Dengan memeriksa seluruh elemen pada satu baris, program dapat mengetahui semua vertex yang dapat dicapai secara langsung dari vertex tersebut.

Cara kerja algoritma dimulai dengan membaca jumlah vertex dan label setiap vertex. Setelah itu program membaca adjacency matrix berukuran $N \times N$ yang berisi informasi hubungan antarvertex. Untuk setiap baris matriks, program mencetak label vertex sebagai vertex asal, kemudian memeriksa seluruh kolom pada baris tersebut. Jika ditemukan nilai yang lebih besar dari 0, berarti terdapat edge menuju vertex pada kolom tersebut sehingga program mencetak pasangan berupa label vertex tujuan dan bobot edge dalam format (vertex,bobot). Sebuah variabel boolean digunakan untuk menentukan apakah vertex tersebut memiliki edge keluar atau tidak. Jika setelah seluruh kolom diperiksa tidak ditemukan nilai positif, maka program mencetak tanda "-" sesuai format keluaran yang diminta oleh soal.

Program ini dibuat berdasarkan beberapa asumsi yang telah dijamin pada soal. Seluruh label vertex bersifat unik sehingga tidak terjadi ambiguitas saat menampilkan adjacency list. Nilai diagonal utama matriks selalu bernilai 0 karena tidak terdapat self-loop. Selain itu, setiap nilai positif pada matriks merepresentasikan bobot edge, sedangkan nilai 0 menunjukkan tidak adanya edge. Graph yang digunakan juga bersifat berarah dan berbobot sehingga hubungan antarvertex hanya ditampilkan sesuai arah yang diberikan pada matriks.

Dari segi kompleksitas, proses pembacaan label vertex membutuhkan waktu $O(N)$. Selanjutnya pembacaan adjacency matrix membutuhkan waktu $O(N^2)$ karena seluruh elemen matriks harus dibaca satu per satu. Proses utama algoritma juga melakukan penelusuran terhadap seluruh elemen matriks untuk menentukan adjacency list setiap vertex, sehingga memerlukan kompleksitas waktu $O(N^2)$. Dengan demikian, kompleksitas waktu total algoritma adalah $O(N^2)$. Untuk kompleksitas ruang, program menyimpan label vertex sebanyak N elemen dan adjacency matrix berukuran $N \times N$, sehingga kebutuhan ruang didominasi oleh matriks dan menghasilkan kompleksitas ruang sebesar $O(N^2)$.

Berdasarkan permasalahan yang diberikan, algoritma ini dapat dibuktikan benar karena setiap elemen matriks diperiksa tepat satu kali. Jika suatu elemen matriks pada posisi (i, j) bernilai lebih besar dari 0, maka program akan menambahkan vertex tujuan ke dalam adjacency list vertex asal beserta bobot yang sesuai. Sebaliknya, jika nilainya 0, maka tidak ada edge yang dicetak. Karena seluruh kolom pada setiap baris diperiksa secara menyeluruh,

maka seluruh edge yang terdapat pada adjacency matrix pasti akan muncul pada adjacency list. Selain itu, apabila suatu vertex tidak memiliki edge keluar, program akan mencetak tanda "-" sesuai ketentuan soal. Dengan demikian, adjacency list yang dihasilkan selalu merepresentasikan graph yang sama dengan adjacency matrix yang diberikan pada input sehingga algoritma terbukti benar untuk menyelesaikan permasalahan ini.

Soal 3: BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS). Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

SR SC

FR FC

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Batasan

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < C$
- $0 \leq FR < R$
- $0 \leq FC < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0	16

0 0	
7 9	

A. Source Code

Table 3 Source Code soal3.cpp

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  int main() {
7      int R, C;
8      cin >> R >> C;
9
10     vector<vector<int>> grid(R, vector<int>(C));
11     for (int i = 0; i < R; i++)
12         for (int j = 0; j < C; j++)
13             cin >> grid[i][j];
14
15     int sr, sc, fr, fc;
16     cin >> sr >> sc >> fr >> fc;
17
18     vector<vector<bool>> visited(R, vector<bool>(C,
19 false));
20     queue<pair<int,int>> q;
21
22     visited[sr][sc] = true;
23     q.push({sr, sc});
24
25     vector<vector<int>> dist(R, vector<int>(C, -1));
26     dist[sr][sc] = 0;
27
28     int dr[] = {-1, 1, 0, 0};
29     int dc[] = {0, 0, -1, 1};
30
31     while (!q.empty()) {
32         int r = q.front().first;
33         int c = q.front().second;
34         q.pop();
35
36         if (r == fr && c == fc) {
37             cout << dist[r][c] << endl;
38             return 0;
39         }
40
41         for (int d = 0; d < 4; d++) {

```

```

42         int nr = r + dr[d];
43         int nc = c + dc[d];
44
45         if (nr >= 0 && nr < R && nc >= 0 && nc < C
46             && !visited[nr][nc] && grid[nr][nc] == 0)
47         {
48             visited[nr][nc] = true;
49             dist[nr][nc] = dist[r][c] + 1;
50             q.push({nr, nc});
51         }
52     }
53 }
54
55 cout << -1 << endl;
56 return 0;
57 }

```

B. Output Program

```

putri@Putris-MacBook-Pro module-6-graph-and-tree-2ZURE % g++ -o soal3 soal3.cpp
putri@Putris-MacBook-Pro module-6-graph-and-tree-2ZURE % ./soal3
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
putri@Putris-MacBook-Pro module-6-graph-and-tree-2ZURE %

```

Gambar 3 Screenshot Hasil Output Soal 3

C. Pembahasan

Algoritma yang digunakan pada program ini adalah **Breadth-First Search (BFS)** untuk mencari jarak atau jumlah langkah minimum dari posisi awal menuju posisi tujuan pada sebuah labirin. BFS merupakan algoritma penelusuran graph yang bekerja dengan mengunjungi seluruh simpul yang berada pada tingkat (level) yang sama terlebih dahulu sebelum berpindah ke tingkat berikutnya. Karena setiap perpindahan pada labirin memiliki biaya yang sama, yaitu satu langkah, BFS sangat cocok digunakan untuk mencari jalur

terpendek pada graph tak berbobot atau graph dengan bobot seragam. Sesuai dengan ketentuan soal, pergerakan hanya diperbolehkan ke empat arah, yaitu atas, bawah, kiri, dan kanan, serta hanya dapat melewati sel yang bernilai 0.

Teknik yang digunakan pada program ini adalah Breadth-First Search menggunakan queue. Struktur data queue digunakan untuk menyimpan posisi-posisi yang akan diproses berdasarkan urutan kedatangannya (First In First Out/FIFO). Dengan mekanisme ini, setiap posisi yang memiliki jarak lebih dekat dari titik awal akan diproses terlebih dahulu dibandingkan posisi yang lebih jauh. Selain itu, program menggunakan matriks visited untuk menandai sel yang sudah pernah dikunjungi agar tidak diproses berulang kali, serta matriks dist untuk menyimpan jumlah langkah minimum dari titik awal menuju setiap sel yang berhasil dicapai.

Cara kerja algoritma dimulai dengan membaca ukuran labirin, isi grid, serta posisi awal dan posisi tujuan. Selanjutnya posisi awal dimasukkan ke dalam queue dan ditandai sebagai telah dikunjungi. Nilai jarak pada posisi awal diinisialisasi dengan 0 karena belum melakukan perpindahan apa pun. Program kemudian menjalankan proses BFS selama queue masih berisi elemen. Pada setiap iterasi, posisi terdepan diambil dari queue dan diperiksa apakah posisi tersebut merupakan tujuan. Jika ya, program langsung mencetak jarak yang tersimpan pada matriks dist dan menghentikan eksekusi. Jika belum mencapai tujuan, program akan memeriksa empat kemungkinan arah pergerakan. Apabila posisi baru masih berada dalam batas grid, belum pernah dikunjungi, dan merupakan jalan yang dapat dilewati (bernilai 0), maka posisi tersebut ditandai sebagai telah dikunjungi, jaraknya diperbarui menjadi jarak posisi sebelumnya ditambah satu, lalu dimasukkan ke dalam queue untuk diproses pada iterasi berikutnya. Jika seluruh posisi yang mungkin telah diproses tetapi tujuan tidak pernah ditemukan, maka program mencetak nilai -1 yang menandakan bahwa tujuan tidak dapat dicapai.

Program dibuat berdasarkan beberapa asumsi yang telah dijamin oleh soal. Ukuran labirin berada pada rentang yang telah ditentukan, setiap sel hanya bernilai 0 atau 1, posisi awal dan tujuan selalu berada di dalam batas grid, serta posisi awal dan tujuan dijamin berada pada sel

yang dapat dilewati. Selain itu, pergerakan hanya diperbolehkan ke empat arah dan algoritma yang digunakan wajib BFS. Dengan adanya jaminan tersebut, program dapat langsung menjalankan proses pencarian tanpa perlu melakukan validasi tambahan terhadap input.

Dari segi kompleksitas, setiap sel pada labirin paling banyak akan dimasukkan ke dalam queue satu kali karena setelah dikunjungi akan ditandai pada matriks visited. Setiap sel juga hanya diperiksa terhadap empat kemungkinan arah pergerakan. Oleh karena itu, kompleksitas waktu algoritma adalah sebesar $O(R \times C)$, dengan R menyatakan jumlah baris dan C menyatakan jumlah kolom pada grid. Kompleksitas ruang juga sebesar $O(R \times C)$ karena program menyimpan matriks visited, matriks dist, dan queue yang dalam kasus terburuk dapat berisi seluruh sel yang dapat dilewati.

Berdasarkan permasalahan yang diberikan, algoritma ini dapat dibuktikan benar karena BFS selalu memproses simpul berdasarkan urutan jaraknya dari titik awal. Ketika suatu sel pertama kali dikunjungi, jarak yang tersimpan pada matriks dist merupakan jarak minimum menuju sel tersebut. Hal ini terjadi karena BFS mengunjungi seluruh posisi yang berjarak k langkah terlebih dahulu sebelum memproses posisi yang berjarak $k + 1$ langkah. Akibatnya, ketika posisi tujuan pertama kali ditemukan, nilai jarak yang tersimpan pasti merupakan jumlah langkah minimum yang diperlukan untuk mencapai tujuan. Jika BFS selesai dijalankan dan tujuan tidak pernah ditemukan, berarti tidak ada jalur yang menghubungkan posisi awal dengan posisi tujuan sehingga output yang benar adalah -1. Dengan demikian, algoritma BFS yang digunakan pada program ini terbukti benar untuk menentukan jarak terpendek keluar labirin sesuai ketentuan soal.

Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

SR SC

FR FC

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.

- $(F R, F C)$ adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Batasan

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < C$
- $0 \leq FR < R$
- $0 \leq FC < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0	4

0 0	
4 5	

A. Source Code

Table 4 Source Code soal4.cpp

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  using namespace std;
5
6  int main() {
7      int N;
8      cin >> N;
9
10     vector<string> labels(N);
11     for (int i = 0; i < N; i++) {
12         cin >> labels[i];
13     }
14
15     vector<vector<int>> matrix(N, vector<int>(N));
16     for (int i = 0; i < N; i++) {
17         for (int j = 0; j < N; j++) {
18             cin >> matrix[i][j];
19         }
20     }
21
22     cout << "Adjacency List:" << endl;
23     for (int i = 0; i < N; i++) {
24         cout << labels[i] << " -> ";
25
26         bool hasEdge = false;
27         bool first = true;
28         for (int j = 0; j < N; j++) {
29             if (matrix[i][j] > 0) {
30                 if (!first) cout << ", ";
31                 cout << "(" << labels[j] << ", " <<
32 matrix[i][j] << ")";
33                 first = false;
34                 hasEdge = true;
35             }
36         }
37
38         if (!hasEdge) {
39             cout << "-";
40         }
41         cout << endl;

```

```

42     }
43
44     return 0;
45 }

```

B. Output Program

```

putri@Putris-MacBook-Pro module-6-graph-and-tree-2ZURE % g++ -o soal4 soal4.cpp
putri@Putris-MacBook-Pro module-6-graph-and-tree-2ZURE % ./soal4
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
putri@Putris-MacBook-Pro module-6-graph-and-tree-2ZURE %

```

Gambar 4 Screenshot Hasil Output Soal 4

C. Pembahasan

Algoritma yang digunakan pada program ini adalah Depth-First Search (DFS) rekursif dengan backtracking untuk menghitung seluruh jalur sederhana dari posisi awal menuju posisi tujuan pada sebuah labirin. DFS merupakan algoritma penelusuran graph yang bekerja dengan menjelajahi satu jalur sedalam mungkin sebelum kembali (backtrack) dan mencoba jalur lainnya. Pada permasalahan ini, tujuan utama bukan mencari jalur terpendek, melainkan menghitung seluruh kemungkinan jalur yang valid. Oleh karena itu, DFS menjadi pilihan yang tepat karena mampu mengeksplorasi setiap kemungkinan rute secara sistematis hingga mencapai tujuan atau menemui jalan buntu. Sesuai ketentuan soal, pergerakan hanya diperbolehkan ke empat arah, yaitu atas, bawah, kiri, dan kanan, serta sebuah jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.

Teknik yang digunakan dalam program ini adalah DFS rekursif dengan mekanisme backtracking. Struktur data visited digunakan untuk menandai sel yang sedang berada dalam jalur yang sedang dieksplorasi. Ketika DFS berpindah ke suatu sel, sel tersebut ditandai

sebagai telah dikunjungi agar tidak dikunjungi kembali dalam jalur yang sama. Setelah seluruh kemungkinan jalur dari sel tersebut selesai diperiksa, status visited dikembalikan menjadi false. Proses inilah yang disebut backtracking, yaitu mengembalikan kondisi ke keadaan sebelumnya agar cabang jalur lain dapat dicoba.

Cara kerja algoritma dimulai dengan membaca ukuran labirin, isi grid, posisi awal, dan posisi tujuan. Selanjutnya program menginisialisasi matriks visited serta variabel totalPaths yang berfungsi untuk menghitung jumlah jalur valid yang ditemukan. Posisi awal ditandai sebagai telah dikunjungi, kemudian fungsi dfs() dipanggil dari posisi tersebut. Pada setiap pemanggilan fungsi, program terlebih dahulu memeriksa apakah posisi saat ini sama dengan posisi tujuan. Jika ya, berarti satu jalur valid telah ditemukan sehingga variabel totalPaths ditambah satu dan fungsi langsung kembali ke pemanggilan sebelumnya. Jika belum mencapai tujuan, program akan memeriksa empat kemungkinan arah pergerakan. Apabila posisi baru masih berada di dalam batas grid, belum pernah dikunjungi, dan merupakan jalan yang dapat dilewati (bernilai 0), maka posisi tersebut ditandai sebagai telah dikunjungi dan DFS dipanggil secara rekursif untuk melanjutkan pencarian. Setelah proses rekursif selesai, status kunjungan pada sel tersebut dikembalikan menjadi false sehingga jalur lain masih dapat menggunakan sel tersebut. Setelah seluruh kemungkinan jalur selesai dieksplorasi, program mencetak nilai totalPaths sebagai jumlah seluruh jalur sederhana dari start menuju finish.

Program dibuat berdasarkan beberapa asumsi yang telah dijamin oleh soal. Setiap sel hanya bernilai 0 atau 1, posisi awal dan tujuan selalu berada pada sel yang dapat dilewati, serta ukuran grid maksimum hanya 7×7 . Selain itu, jumlah jalur valid dijamin tidak melebihi 100000 dan algoritma yang digunakan wajib DFS rekursif dengan backtracking. Adanya batasan ukuran grid yang relatif kecil membuat eksplorasi seluruh kemungkinan jalur masih dapat dilakukan dalam batas waktu yang tersedia.

Dari segi kompleksitas, algoritma DFS pada kasus ini memiliki kompleksitas waktu yang bergantung pada jumlah kemungkinan jalur yang dapat dibentuk. Dalam kasus terburuk, setiap sel dapat memiliki beberapa pilihan arah sehingga jumlah kemungkinan jalur dapat

bertumbuh secara eksponensial terhadap jumlah sel yang dapat dilalui. Oleh karena itu, kompleksitas waktu dapat dinyatakan sebagai $O(4^{(R \times C)})$ pada kasus terburuk. Meskipun demikian, batas ukuran grid yang kecil membuat algoritma tetap dapat dijalankan dengan baik. Untuk kompleksitas ruang, program menggunakan matriks visited berukuran $R \times C$ serta tumpukan rekursi (recursion stack) yang dalam kasus terburuk dapat menyimpan seluruh sel pada jalur yang sedang dieksplorasi. Oleh karena itu, kompleksitas ruangnya adalah $O(R \times C)$.

Berdasarkan permasalahan yang diberikan, algoritma ini dapat dibuktikan benar karena DFS akan mencoba seluruh kemungkinan pergerakan yang valid dari posisi awal hingga tujuan. Setiap kali mencapai posisi tujuan, satu jalur valid akan dihitung. Penggunaan matriks visited memastikan bahwa suatu jalur tidak pernah mengunjungi sel yang sama lebih dari satu kali, sehingga seluruh jalur yang dihitung memenuhi definisi jalur sederhana. Selain itu, mekanisme backtracking memungkinkan program untuk kembali ke kondisi sebelumnya dan mengeksplorasi cabang jalur lain yang belum dicoba. Karena seluruh kemungkinan pergerakan yang valid diperiksa secara lengkap dan tidak ada jalur yang dihitung lebih dari satu kali, maka nilai totalPaths yang dihasilkan merupakan jumlah seluruh jalur sederhana dari posisi awal menuju posisi tujuan. Dengan demikian, algoritma DFS dengan backtracking yang digunakan pada program ini terbukti benar untuk menyelesaikan permasalahan yang diberikan.

Soal 5: Tree Traversal

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah Red-Black Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N$

Q

T_1

T_2

...

T_Q

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .

- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Batasan

- $0 \leq N \leq 1000$
- $1000000000 \leq A_i \leq 10000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER	[Preorder] : 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder] : 20 40 30 60 80 70 50
ALL	

A. Source Code

Table 5 Source Code soal5.cpp

1	#include <iostream>
2	#include <string>
3	#include <vector>
4	#include "RedBlackTree.h"
5	using namespace std;

```

6
7 void preorder(const RedBlackTree::Node* node, const
8 RedBlackTree::Node* nil, vector<int>& result) {
9     if (node->isNil) return;
10    result.push_back(node->key);
11    preorder(node->left, nil, result);
12    preorder(node->right, nil, result);
13 }
14
15 void inorder(const RedBlackTree::Node* node, const
16 RedBlackTree::Node* nil, vector<int>& result) {
17     if (node->isNil) return;
18     inorder(node->left, nil, result);
19     result.push_back(node->key);
20     inorder(node->right, nil, result);
21 }
22
23 void postorder(const RedBlackTree::Node* node, const
24 RedBlackTree::Node* nil, vector<int>& result) {
25     if (node->isNil) return;
26     postorder(node->left, nil, result);
27     postorder(node->right, nil, result);
28     result.push_back(node->key);
29 }
30
31 void printResult(const string& label, const vector<int>&
32 result) {
33     cout << label << " :";
34     for (int x : result) {
35         cout << " " << x;
36     }
37     cout << endl;
38 }
39
40 int main() {
41     int N;
42     cin >> N;
43
44     RedBlackTree rbt;
45
46     for (int i = 0; i < N; i++) {
47         int val;
48         cin >> val;
49         // Abaikan duplikat
50         if (!rbt.contains(val)) {
51             rbt.insert(val);
52         }

```



```

53     }
54
55     int Q;
56     cin >> Q;
57
58     vector<string> queries(Q);
59     for (int i = 0; i < Q; i++) {
60         cin >> queries[i];
61     }
62
63     for (const string& query : queries) {
64         if (rbt.empty()) {
65             cout << "Tree kosong. Tidak ada yang bisa
66 ditampilkan." << endl;
67             continue;
68         }
69
70         if (query == "PREORDER") {
71             vector<int> res;
72             preorder(rbt.root(), rbt.nil(), res);
73             printResult("[Preorder]", res);
74         } else if (query == "INORDER") {
75             vector<int> res;
76             inorder(rbt.root(), rbt.nil(), res);
77             printResult("[Inorder]", res);
78         } else if (query == "POSTORDER") {
79             vector<int> res;
80             postorder(rbt.root(), rbt.nil(), res);
81             printResult("[Postorder]", res);
82         } else if (query == "ALL") {
83             vector<int> pre, in, post;
84             preorder(rbt.root(), rbt.nil(), pre);
85             inorder(rbt.root(), rbt.nil(), in);
86             postorder(rbt.root(), rbt.nil(), post);
87             printResult("[Preorder]", pre);
88             printResult("[Inorder]", in);
89             printResult("[Postorder]", post);
90         }
91     }
92
93     return 0;
94 }

```

Table 6 Source Code RedBlackTree.h

1	#ifndef RED_BLACK_TREE_H
2	#define RED_BLACK_TREE_H

```

3
4 #include <cstddef>
5
6 class RedBlackTree {
7     public:
8         enum class Color { RED, BLACK };
9
10        struct Node {
11            int key;
12            Color color;
13            Node* left;
14            Node* right;
15            Node* parent;
16            bool isNil;
17
18            Node(int value, Color nodeColor, bool sentinel);
19        };
20
21    private:
22        Node* rootNode;
23        Node* nilNode;
24        std::size_t nodeCount;
25
26    private:
27        void rotateLeft(Node* node);
28        void rotateRight(Node* node);
29        void fixInsert(Node* node);
30        void destroy(Node* node);
31
32    public:
33        RedBlackTree();
34        ~RedBlackTree();
35
36        RedBlackTree(const RedBlackTree& other) = delete;
37        RedBlackTree& operator=(const RedBlackTree& other) =
38 delete;
39
40        void insert(int key);
41        bool contains(int key) const;
42
43        bool lowerBound(int key, int& result) const;
44        bool upperBound(int key, int& result) const;
45
46        const Node* root() const;
47        const Node* nil() const;
48
49        bool empty() const;

```

50	std::size_t size() const;
51	
52	void clear();
53	};
54	
55	#endif

Table 7 Source Code RedBlackTree.cpp

1	#include "RedBlackTree.h"
2	
3	RedBlackTree::Node::Node(int value, Color nodeColor,
4	bool sentinel)
5	: key(value),
6	color(nodeColor),
7	left(nullptr),
8	right(nullptr),
9	parent(nullptr),
10	isNil(sentinel) {}
11	
12	RedBlackTree::RedBlackTree()
13	: rootNode(nullptr),
14	nilNode(nullptr),
15	nodeCount(0) {
16	nilNode = new Node(0, Color::BLACK, true);
17	nilNode->left = nilNode;
18	nilNode->right = nilNode;
19	nilNode->parent = nilNode;
20	
21	rootNode = nilNode;
22	}
23	
24	RedBlackTree::~~RedBlackTree() {
25	clear();
26	delete nilNode;
27	}
28	
29	void RedBlackTree::rotateLeft(Node* node) {
30	Node* child = node->right;
31	
32	node->right = child->left;
33	if (!child->left->isNil) {
34	child->left->parent = node;
35	}
36	
37	child->parent = node->parent;
38	

```

39     if (node->parent->isNil) {
40         rootNode = child;
41     } else if (node == node->parent->left) {
42         node->parent->left = child;
43     } else {
44         node->parent->right = child;
45     }
46
47     child->left = node;
48     node->parent = child;
49 }
50
51 void RedBlackTree::rotateRight(Node* node) {
52     Node* child = node->left;
53
54     node->left = child->right;
55     if (!child->right->isNil) {
56         child->right->parent = node;
57     }
58
59     child->parent = node->parent;
60
61     if (node->parent->isNil) {
62         rootNode = child;
63     } else if (node == node->parent->right) {
64         node->parent->right = child;
65     } else {
66         node->parent->left = child;
67     }
68
69     child->right = node;
70     node->parent = child;
71 }
72
73 void RedBlackTree::fixInsert(Node* node) {
74     while (node->parent->color == Color::RED) {
75         Node* parent = node->parent;
76         Node* grandparent = parent->parent;
77
78         if (parent == grandparent->left) {
79             Node* uncle = grandparent->right;
80
81             if (uncle->color == Color::RED) {
82                 parent->color = Color::BLACK;
83                 uncle->color = Color::BLACK;
84                 grandparent->color = Color::RED;
85                 node = grandparent;

```

```

86         } else {
87             if (node == parent->right) {
88                 node = parent;
89                 rotateLeft(node);
90             }
91
92             node->parent->color = Color::BLACK;
93             node->parent->parent->color =
94 Color::RED;
95             rotateRight(node->parent->parent);
96         }
97     } else {
98         Node* uncle = grandparent->left;
99
100        if (uncle->color == Color::RED) {
101            parent->color = Color::BLACK;
102            uncle->color = Color::BLACK;
103            grandparent->color = Color::RED;
104            node = grandparent;
105        } else {
106            if (node == parent->left) {
107                node = parent;
108                rotateRight(node);
109            }
110
111            node->parent->color = Color::BLACK;
112            node->parent->parent->color =
113 Color::RED;
114            rotateLeft(node->parent->parent);
115        }
116    }
117 }
118
119 rootNode->color = Color::BLACK;
120 }
121
122 void RedBlackTree::insert(int key) {
123     Node* newNode = new Node(key, Color::RED, false);
124     newNode->left = nilNode;
125     newNode->right = nilNode;
126     newNode->parent = nilNode;
127
128     Node* parent = nilNode;
129     Node* current = rootNode;
130
131     while (!current->isNil) {
132         parent = current;

```

```

133         if (key < current->key) {
134             current = current->left;
135         } else {
136             current = current->right;
137         }
138     }
139 }
140
141 newNode->parent = parent;
142
143 if (parent->isNil) {
144     rootNode = newNode;
145 } else if (key < parent->key) {
146     parent->left = newNode;
147 } else {
148     parent->right = newNode;
149 }
150
151 nodeCount++;
152 fixInsert(newNode);
153 }
154
155 bool RedBlackTree::contains(int key) const {
156     Node* current = rootNode;
157
158     while (!current->isNil) {
159         if (key == current->key) {
160             return true;
161         }
162
163         if (key < current->key) {
164             current = current->left;
165         } else {
166             current = current->right;
167         }
168     }
169
170     return false;
171 }
172
173 bool RedBlackTree::lowerBound(int key, int& result) const
174 {
175     Node* current = rootNode;
176     Node* answer = nilNode;
177
178     while (!current->isNil) {
179         if (current->key >= key) {

```

```

180         answer = current;
181         current = current->left;
182     } else {
183         current = current->right;
184     }
185 }
186
187 if (answer->isNil) {
188     return false;
189 }
190
191 result = answer->key;
192 return true;
193 }
194
195 bool RedBlackTree::upperBound(int key, int& result) const
196 {
197     Node* current = rootNode;
198     Node* answer = nilNode;
199
200     while (!current->isNil) {
201         if (current->key > key) {
202             answer = current;
203             current = current->left;
204         } else {
205             current = current->right;
206         }
207     }
208
209     if (answer->isNil) {
210         return false;
211     }
212
213     result = answer->key;
214     return true;
215 }
216
217 const RedBlackTree::Node* RedBlackTree::root() const {
218     return rootNode;
219 }
220
221 const RedBlackTree::Node* RedBlackTree::nil() const {
222     return nilNode;
223 }
224
225 bool RedBlackTree::empty() const {
226     return nodeCount == 0;

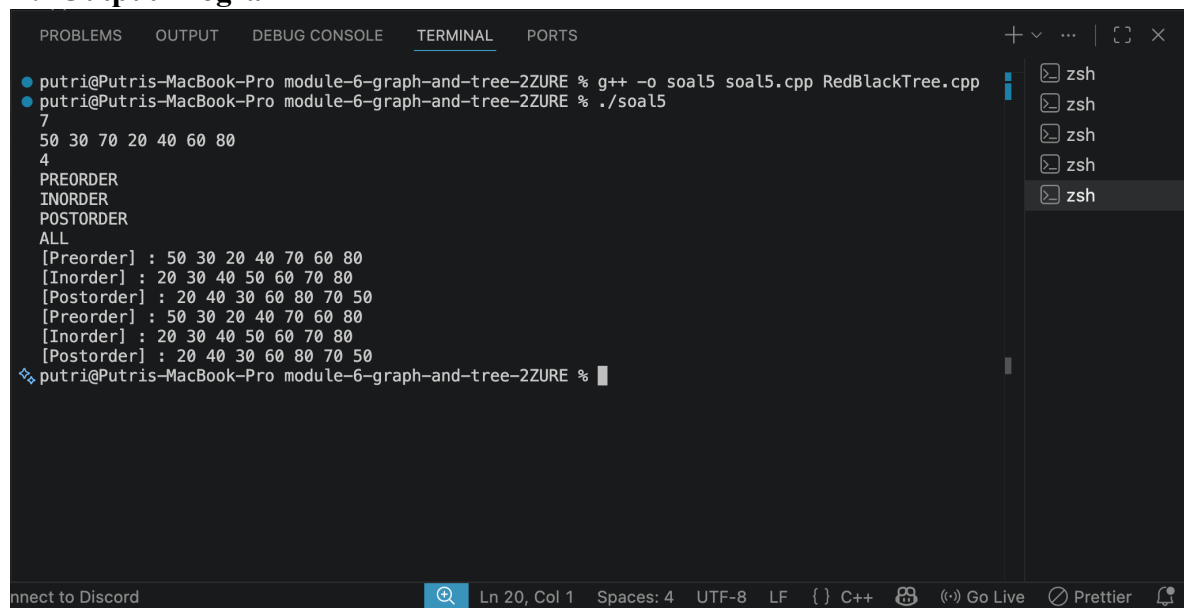
```

```

227 }
228
229 std::size_t RedBlackTree::size() const {
230     return nodeCount;
231 }
232
233 void RedBlackTree::destroy(Node* node) {
234     if (node->isNil) {
235         return;
236     }
237
238     destroy(node->left);
239     destroy(node->right);
240     delete node;
241 }
242
243 void RedBlackTree::clear() {
244     destroy(rootNode);
245     rootNode = nilNode;
246     nodeCount = 0;
247 }

```

B. Output Program



```

putri@Putris-MacBook-Pro module-6-graph-and-tree-2ZURE % g++ -o soal5 soal5.cpp RedBlackTree.cpp
putri@Putris-MacBook-Pro module-6-graph-and-tree-2ZURE % ./soal5
7
4
PREORDER
INORDER
POSTORDER
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
putri@Putris-MacBook-Pro module-6-graph-and-tree-2ZURE %

```

Gambar 5 Screenshot Hasil Output Soal 5

C. Pembahasan

Algoritma yang digunakan pada program ini bertujuan untuk menampilkan hasil traversal dari sebuah Red-Black Tree (RBT) yang dibentuk dari sekumpulan bilangan bulat. Red-Black Tree merupakan struktur data pohon biner pencarian (Binary Search Tree) yang

memiliki aturan tambahan berupa pewarnaan node untuk menjaga keseimbangan pohon. Dengan keseimbangan tersebut, operasi pencarian, penyisipan, dan penghapusan dapat dilakukan dengan efisien. Pada soal ini, seluruh bilangan dimasukkan ke dalam Red-Black Tree sesuai urutan input, kemudian pengguna dapat meminta hasil traversal berupa Preorder, Inorder, Postorder, atau ketiganya sekaligus melalui query ALL. Jika terdapat data duplikat, data tersebut tidak dimasukkan kembali ke dalam tree.

Teknik yang digunakan pada program ini adalah tree traversal secara rekursif. Terdapat tiga fungsi traversal yang dibuat, yaitu `preorder()`, `inorder()`, dan `postorder()`. Ketiga fungsi tersebut bekerja dengan prinsip rekursi, yaitu fungsi memanggil dirinya sendiri untuk mengunjungi subtree kiri dan subtree kanan hingga mencapai node sentinel (`nil`). Pada traversal preorder, node dikunjungi terlebih dahulu sebelum subtree kiri dan kanan. Pada traversal inorder, subtree kiri dikunjungi terlebih dahulu, kemudian node, lalu subtree kanan. Sedangkan pada traversal postorder, subtree kiri dan kanan dikunjungi terlebih dahulu sebelum node diproses.

Cara kerja program dimulai dengan membaca jumlah data yang akan dimasukkan ke dalam Red-Black Tree. Setiap nilai yang dibaca terlebih dahulu diperiksa menggunakan fungsi `contains()`. Jika nilai tersebut belum ada di dalam tree, maka nilai dimasukkan menggunakan fungsi `insert()`. Sebaliknya, jika nilai sudah ada, maka data tersebut diabaikan sesuai ketentuan soal. Setelah seluruh data dimasukkan, program membaca sejumlah query traversal yang diminta pengguna. Untuk setiap query, program terlebih dahulu memeriksa apakah tree kosong. Jika tree kosong, program mencetak pesan bahwa tidak ada data yang dapat ditampilkan. Jika tree tidak kosong, program akan menjalankan traversal sesuai jenis query yang diberikan. Hasil traversal disimpan ke dalam sebuah vector, kemudian dicetak menggunakan fungsi `printResult()`. Untuk query ALL, program menjalankan ketiga traversal sekaligus dan menampilkan seluruh hasilnya secara berurutan.

Program dibuat berdasarkan beberapa asumsi yang telah dijamin oleh soal. Jumlah data maksimum adalah 1000 elemen dan jumlah query maksimum adalah 20. Jenis query yang diberikan selalu berupa PREORDER, INORDER, POSTORDER, atau ALL. Selain itu,

struktur Red-Black Tree yang digunakan telah menjamin bahwa seluruh operasi penyisipan dilakukan sesuai aturan Red-Black Tree sehingga pohon tetap seimbang setelah setiap proses insert. Data duplikat juga diabaikan sehingga setiap nilai hanya muncul satu kali dalam tree. Dari segi kompleksitas, proses penyisipan setiap elemen ke dalam Red-Black Tree memiliki kompleksitas waktu $O(\log N)$ karena tinggi pohon selalu terjaga seimbang. Jika terdapat N data yang dimasukkan, maka proses pembentukan tree membutuhkan waktu sebesar $O(N \log N)$. Selanjutnya, setiap operasi traversal akan mengunjungi seluruh node tepat satu kali sehingga memiliki kompleksitas waktu sebesar $O(N)$. Jika query yang diminta adalah ALL, maka ketiga traversal dijalankan secara terpisah sehingga kompleksitasnya tetap berada pada orde $O(N)$. Untuk kompleksitas ruang, fungsi traversal menggunakan vector untuk menyimpan hasil traversal sebanyak N elemen dan memanfaatkan recursion stack yang maksimum sebesar tinggi pohon. Karena tinggi Red-Black Tree adalah $O(\log N)$, maka kebutuhan ruang keseluruhan adalah $O(N)$.

Berdasarkan permasalahan yang diberikan, algoritma ini dapat dibuktikan benar karena setiap fungsi traversal mengikuti definisi traversal pohon biner yang baku. Fungsi preorder selalu mengunjungi root terlebih dahulu sebelum subtree kiri dan subtree kanan sehingga menghasilkan urutan root-left-right. Fungsi inorder selalu mengunjungi subtree kiri, kemudian root, lalu subtree kanan sehingga menghasilkan urutan left-root-right. Pada Binary Search Tree maupun Red-Black Tree, traversal inorder selalu menghasilkan data dalam urutan menaik. Sementara itu, fungsi postorder mengunjungi subtree kiri dan kanan terlebih dahulu sebelum root sehingga menghasilkan urutan left-right-root. Karena setiap node dikunjungi tepat satu kali dan urutan kunjungan sesuai definisi masing-masing traversal, maka hasil yang dicetak program pasti sesuai dengan traversal yang diminta pada query. Oleh karena itu, algoritma ini terbukti benar untuk menyelesaikan permasalahan tree traversal pada Red-Black Tree sesuai spesifikasi soal.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-2ZURE>